

Estructuras de datos

- Se compara una **lista de compras** con las **estructuras de datos en Python**.
- **Listas (`list`)**: almacenan colecciones ordenadas (ej. `["manzana", "plátano"]`); permiten añadir, eliminar o modificar elementos, incluso contener listas dentro de listas.
- **Diccionarios (`dict`)**: permiten asociar claves con valores (ej. `{"manzana": 1.5, "aguacate": 2.0}`); útiles para buscar información por nombre, como precios.
- **Conjuntos (`set`)**: almacenan elementos únicos (sin duplicados); sirven para comprobar si ya tienes un artículo o combinar colecciones y encontrar coincidencias.
- La idea central: **cada estructura tiene su propósito**, como organizar alimentos en una cesta. Elegir la adecuada hace el código más claro, eficiente y mantenible.

En conclusión: listas, diccionarios y sets son las “cestas” de Python para organizar datos, y usarlas bien es como tener una compra ordenada y sin desperdicios.

Estructuras de datos en Python: Listas y Tuplas

Listas (`list`)

- Colecciones **ordenadas y mutables**.
- Pueden contener **diferentes tipos de datos** (números, cadenas, listas, etc.).
- Se accede a los elementos con **índices (empezando en 0)**.
- Soportan operaciones de **slicing** para obtener subconjuntos.
- Se pueden **modificar dinámicamente**: añadir, eliminar, reordenar.

Operaciones comunes:

- `append()`: añadir un elemento.

- `extend()`: añadir varios elementos de otra lista.
- `insert()`: insertar en una posición específica.
- `remove()` y `pop()`: eliminar elementos.
- `sort()` y `reverse()`: reordenar la lista.

Ejemplos:

```
# Crear una lista con distintos tipos de datos
```

```
my_list = [1, 2.1, "hello", [3, 4]]
```

```
# Acceso por índice
```

```
print(my_list[0])    # 1
```

```
print(my_list[1])    # 2.1
```

```
# Slicing
```

```
print(my_list[1:3])  # [2.1, "hello"]
```

```
# Extender una lista
```

```
tasks = ["Take a nap"]
```

```
new_tasks = ["Go for a walk", "Read a book"]
```

```
tasks.extend(new_tasks)  # ["Take a nap", "Go for a walk", "Read a book"]
```

```
# Aplicación práctica: lista de tareas
```

```
tasks = ["Buy groceries", "Finish report", "Call Mom"]
```

```
tasks.append("Programar cita con el dentista")
```

```
tasks.remove("Finish report")
```

```
tasks.sort()
```

```
print(tasks)
```

Tuplas (**tuple**)

- Similares a las listas, pero **inmutables** (no se pueden modificar una vez creadas).
- Se definen con **paréntesis** en lugar de corchetes.
- También permiten **indexación y slicing**.

- Garantizan la **integridad de los datos** → ideales para información que no debe cambiar.

Ejemplos:

```
# Crear una tupla
my_tuple = (10, 20, "python")

# Acceso por índice
print(my_tuple[2])    # "python"

# Slicing
print(my_tuple[0:2])  # (10, 20)

# Intentar modificar dará error
# my_tuple[1] = 30 # ✗Error
# my_tuple.append("nuevo elemento") # ✗Error

# Aplicación práctica: coordenadas geográficas
coordinates = (37.7749, -122.4194) # Latitud y longitud de San Francisco
```

Idea clave:

- Usa **listas** cuando necesites colecciones dinámicas que cambien con el tiempo.
- Usa **tuplas** para datos **fijos y seguros** que no deben alterarse.

Conjuntos (**set**)

- Colecciones **desordenadas y sin duplicados**.
- No permiten acceder por **índice**.
- Útiles para operaciones matemáticas de conjuntos: unión, intersección, diferencia.

Ejemplo básico:

```
# Creación de un conjunto con duplicados
my_set = {1, 2, 3, 3}
```

```
print(my_set)    # {1, 2, 3} -> el duplicado se elimina
```

```
# Operaciones con conjuntos
```

```
a = {1, 2, 3}
```

```
b = {3, 4, 5}
```

```
a.add(6)          # Añadir elemento
```

```
a.remove(2)       # Eliminar elemento
```

```
print(a.union(b))  # {1, 3, 4, 5, 6}
```

```
print(a.intersection(b)) # {3}
```

```
print(a.difference(b)) # {1, 6}
```

Aplicación real: Seguimiento de likes en redes sociales

```
liked_by = {12345, 67890, 98765} # IDs únicos de usuarios
```

```
print(len(liked_by)) # 3 usuarios únicos
```

Diccionarios (**dict**)

- Estructuras basadas en **pares clave-valor**.
- Cada clave es única y apunta a un valor.
- Son **mutables y dinámicos** → se pueden añadir, actualizar o eliminar datos.

Ejemplo básico:

```
# Creación de un diccionario
```

```
my_dict = {"name": "Alice", "age": 30, "city": "New York"}
```

```
# Acceso por clave
```

```
print(my_dict["age"]) # 30
```

```
# Métodos útiles
```

```
print(my_dict.get("city")) # "New York"
```

```
print(my_dict.keys())      # dict_keys(['name', 'age', 'city'])
```

```
print(my_dict.values())    # dict_values(['Alice', 30, 'New York'])
```

```
print(my_dict.items())     #
```

```
dict_items([('name', 'Alice'), ('age', 30), ('city', 'New York')])
```

```
# Modificación
my_dict.update({"age": 31})
print(my_dict["age"]) # 31
```

Aplicación real: Catálogo de productos

```
products = {
    "P101": {"name": "Laptop", "price": 999.99},
    "P102": {"name": "Smartphone", "price": 599.99}
}

# Acceso al precio de un producto
print(products["P101"]["price"]) # 999.99
```

Conclusión general:

- **Sets** → ideales para colecciones **únicas** y operaciones matemáticas.
- **Dicts** → perfectos para datos estructurados con **claves** y recuperación rápida de información.

Mutabilidad en Python

La **mutabilidad** indica si una estructura de datos puede **modificarse después de creada**. En Python tenemos contenedores mutables e inmutables:

- **Listas (**list**)** → **mutables**, ordenadas, permiten añadir, eliminar y modificar elementos.
- **Diccionarios (**dict**)** → **mutables**, pares clave-valor dinámicos.
- **Tuplas (**tuple**)** → **inmutables**, su contenido no puede cambiar.
- **Conjuntos (**set**)** → **mutables**, pero sólo almacenan elementos únicos y sin orden.

Listas (mutables)

- Pueden almacenar cualquier tipo de dato.
- Métodos útiles:
 - `.append()` → añadir elementos.
 - `.remove()` o `.pop()` → eliminar.
 - `.sort()` → ordenar.
 - `.index()` → buscar.
 - Slicing → cortar o reemplazar.

Ejemplo:

```
shopping_list = ["apples", "bananas", "milk"]

# Añadir un ítem
shopping_list.append("eggs")

# Eliminar un ítem
shopping_list.remove("apples")

print(shopping_list)
# ['bananas', 'milk', 'eggs']
```

Diccionarios (mutables)

- Almacenan datos como **clave** → **valor**.
- Permiten añadir, actualizar o eliminar pares.
- Valores pueden ser de cualquier tipo (listas, otros diccionarios, etc.).

Ejemplo:

```
item_quantities = {"apples": 3, "bananas": 1}
```

```
# Añadir un ítem con su cantidad
item_quantities["eggs"] = 12

# Actualizar cantidad
item_quantities["bananas"] += 2

# Eliminar un ítem
del item_quantities["apples"]

print(item_quantities)
# {'bananas': 3, 'eggs': 12}
```

Ejemplo práctico completo: App de compras

```
shopping_list = ["apples", "bananas", "milk"] # Lista de ítems
item_quantities = {"apples": 3, "bananas": 1} # Diccionario con
cantidades

# Usuario añade un ítem
shopping_list.append("eggs")
item_quantities["eggs"] = 12

# Usuario aumenta la cantidad de bananas
item_quantities["bananas"] += 2

# Usuario elimina apples
shopping_list.remove("apples")
del item_quantities["apples"]

# Resultados
print(shopping_list)      # ['bananas', 'milk', 'eggs']
print(item_quantities)    # {'bananas': 3, 'eggs': 12}
```

Conclusión:

- **Listas y diccionarios son mutables** → ideales para programas dinámicos donde los datos cambian con frecuencia.

- La mutabilidad permite que el programa se adapte a entradas de usuario y condiciones cambiantes.

Tuplas y Conjuntos

Tuplas → Inmutables

- Una vez creadas, **no se pueden modificar** (no añadir, eliminar ni cambiar elementos).
- Útiles para **datos constantes** como coordenadas, fechas o configuraciones.
- Pueden usarse como **claves en diccionarios**, gracias a su inmutabilidad.

Ejemplos:

```
# Tuplas: datos fijos
coordinates = (37.7749, -122.4194) # Latitud y longitud de San Francisco
birth_date = (1990, 12, 25)      # Año, mes, día

# Acceso a elementos
print(coordinates[0]) # 37.7749
print(birth_date[1])  # 12
```

Conjuntos → Mutables (por defecto)

- Colecciones de elementos **únicos y desordenados**.
- Se pueden **añadir** o **eliminar** elementos (`.add()`, `.remove()`).
- Eliminar duplicados convirtiendo una lista en un conjunto.
- Existe un tipo especial: **frozenset** (inmutable).

Ejemplos:

```
# Conjunto básico
```

```
unique_colors = {"red", "green", "blue"}

# Eliminar duplicados de una lista
numbers = [1, 2, 2, 3, 4, 4, 5]
unique_numbers = set(numbers)

print(unique_numbers) # {1, 2, 3, 4, 5}

# Operaciones de conjuntos
A = {1, 2, 3}
B = {3, 4, 5}
print(A.union(B))      # {1, 2, 3, 4, 5}
print(A.intersection(B)) # {3}
print(A.difference(B))  # {1, 2}
```

Ventajas de la Inmutabilidad

- Datos más **seguros y predecibles** (no cambian accidentalmente).
- Python puede **optimizar almacenamiento y velocidad**.
- Tipos inmutables como `tuple` o `frozenset` se pueden usar como **claves en diccionarios**.

Mutable vs Inmutable

Mutable (listas, diccionarios, conjuntos)

- Flexibles y dinámicos.
- Útiles cuando los datos **cambian con frecuencia**.
- Ejemplo:

```
# Lista de tareas (mutable)
tasks = ["Buy groceries", "Finish report"]
tasks.append("Call mom")
print(tasks) # ['Buy groceries', 'Finish report', 'Call mom']
```

```
# Diccionario con datos actualizables
stock_prices = {"AAPL": 175, "GOOG": 2800}
stock_prices["AAPL"] = 180 # actualizar valor
print(stock_prices) # {'AAPL': 180, 'GOOG': 2800}
```

Inmutables (tuplas, frozenset)

- Protegen datos importantes contra cambios accidentales.
- Útiles para **coordenadas, identificadores, configuración, claves de diccionarios**.
- Ejemplo:

```
# frozenset inmutable
fs = frozenset([1, 2, 3])
print(fs) # frozenset({1, 2, 3})

# Se puede usar como clave en un diccionario
data = {fs: "grupo de números"}
print(data[fs]) # grupo de números
```

Conclusión

- Usa **mutables** (listas, diccionarios, sets) cuando los datos cambian a menudo.
- Usa **inmutables** (tuplas, frozenset) cuando la **integridad y seguridad de los datos** es esencial.
- La clave está en **elegir la estructura adecuada según el contexto**.

Diccionarios en Python

Un **diccionario** es una estructura de datos en Python que organiza la información en **pares clave-valor**, como una **libreta de direcciones digital**:

- **Clave** → identificador único (ej. nombre, código de producto).
- **Valor** → información asociada (ej. número de teléfono, precio, dirección).

Características principales

1. Búsquedas eficientes

- Permiten recuperar valores rápidamente a partir de una clave.
- Muy útiles en aplicaciones como **motores de búsqueda, traductores o tiendas online**.

2. Mutabilidad

- Se pueden **añadir, eliminar o modificar** pares clave-valor en cualquier momento.
- Ideales para datos que cambian con frecuencia, como:
 - Catálogo de productos (precios, stock).
 - Redes sociales (perfiles, publicaciones).

3. Modelado de relaciones

- Representan bien **entidades del mundo real** con sus atributos.
- Ejemplo: un **CRM** puede usar un diccionario por cliente, con información como:
 - Contacto, historial de compras, soporte.

Ejemplos de uso

Base de datos de empleados:

```
empleado = {  
    "nombre": "Ana Pérez",  
    "departamento": "Marketing"  
}
```

-

Catálogo de productos online:

```
productos = {  
    "P101": {"nombre": "Laptop", "precio": 999.99},  
    "P102": {"nombre": "Smartphone", "precio": 599.99}  
}
```

-

Conclusión

Los **diccionarios** son esenciales en Python porque:

- Hacen que el acceso a los datos sea **rápido y eficiente**.
- Son **flexibles y mutables**, adaptándose a cambios constantes.
- Modelan relaciones complejas del mundo real de manera estructurada.

Con práctica (usando **diccionarios anidados** o **comprensiones**), puedes aprovechar aún más su poder en proyectos y aplicaciones.

Pregunta

Imagina que estás creando un programa para almacenar información de contacto. Necesitas encontrar rápidamente el número de teléfono de una persona utilizando su nombre. ¿Cuál de las siguientes opciones sería la forma MÁS eficaz de almacenar y recuperar esta información? Selecciona la mejor respuesta.

- ☒ Un diccionario donde el nombre de la persona es la clave y su número de teléfono es el valor.
- ☐ Una lista en la que cada elemento contiene el nombre y el número de teléfono de la persona. Tendrías que repasar la lista para encontrar el nombre correcto.
- ☐ Listas separadas para nombres y números de teléfono, donde la posición en cada lista corresponde a la misma persona.
- ☐ Un simple archivo de texto en el que cada línea contiene un nombre y un número de teléfono, y que se lee a través del archivo para encontrar el nombre.

☒ **Correcto**

Correcto Al igual que la "libreta de direcciones de alta tecnología" descrita en el vídeo, un diccionario permite acceder directamente al número de teléfono (el valor) utilizando el nombre de la persona (la clave única). Es la forma más eficaz de buscar información sin tener que buscar en una secuencia de elementos.

Diccionarios en Python: aplicaciones reales

¿Qué son los diccionarios?

- Son estructuras de datos que almacenan **pares clave-valor**.
- Funcionan como un **diccionario real**: la palabra (clave) da acceso a su significado (valor).
- Las **claves** deben ser únicas y pueden ser números, cadenas o tuplas.
- Los **valores** pueden ser cualquier tipo de dato: números, texto, listas, objetos, otros diccionarios.
- Desde Python 3.7, **mantienen el orden de inserción**.

Ventajas principales

1. **Almacenamiento y recuperación rápida de datos**.
2. **Organización estructurada** y más legible del código.
3. **Flexibilidad** → se pueden anidar para crear estructuras complejas (árboles, gráficos).
4. **Actualización dinámica** de datos: modificar valores de claves existentes sin duplicación.

Escenarios reales de uso

Libro de calificaciones

```
calificaciones = {"Ana": 95, "Luis": 88, "María": 76}
```

1. → Permite consultar notas, calcular promedios o identificar mejores resultados.

Perfiles en redes sociales

```
perfil = {  
    "usuario": "juan23",  
    "nombre": "Juan Pérez",  
    "edad": 25,
```

```
"intereses": ["música", "programación"],  
"amigos": ["ana99", "luis88"]  
}
```

2. →Representa información completa del usuario (datos, amigos, mensajes, configuración).

Catálogo de productos en una tienda online

```
productos = {  
    "P101": {"nombre": "Libro Python", "precio": 29.99, "stock":  
15},  
    "P102": {"nombre": "Auriculares", "precio": 59.99, "stock": 8}  
}
```

3. →Facilita búsquedas, gestión de inventario y filtrado de artículos.

Traducción de idiomas

```
traducciones = {"hello": "hola", "goodbye": "adiós"}
```

4. →Base para apps de traducción.

Archivos de configuración

```
configuracion = {  
    "volumen": 80,  
    "resolucion": "1920x1080",  
    "dificultad": "media"  
}
```

5. →Guarda ajustes de usuario en juegos o aplicaciones.

Contar frecuencia de palabras en un texto

```
texto = "hola hola python"  
frecuencia = {"hola": 2, "python": 1}
```

6. →Muy útil en **procesamiento de lenguaje natural**.

Retos a considerar

- **Claves duplicadas no existen** → si se repite, sobrescribe el valor anterior.
- **Antes de Python 3.7 no se garantizaba el orden**, pero ahora sí.

Conclusión

Los **diccionarios** son:

Rápidos en búsquedas.

Flexibles y dinámicos.

Capaces de modelar datos simples y complejos.

Son una **piedra angular en Python** y dominarlos permite crear aplicaciones potentes: desde simples scripts hasta sistemas complejos como redes sociales, motores de búsqueda o procesadores de lenguaje natural.

Conjuntos en Python: Elementos únicos y operaciones eficientes

Un **set** en Python es una colección **desordenada de elementos únicos**. Cada elemento solo aparece una vez, lo que los hace ideales para:

- Eliminar duplicados automáticamente.
- Comprobar la pertenencia de un elemento de manera rápida.
- Consolidar datos de múltiples fuentes sin repeticiones.

Propiedades clave de los sets

1. **Unicidad:**

Cada elemento es único; si se intenta añadir un duplicado, se descarta automáticamente.

2. Desordenados:

No mantienen un orden específico, por lo que no se accede a elementos mediante índices.

3. Mutables (por defecto):

Puedes añadir (`add()`) o eliminar (`remove()`) elementos.

Existe `frozenset` para sets inmutables.

4. Pruebas de pertenencia eficientes:

La verificación de si un elemento está en el set es muy rápida, incluso en grandes conjuntos de datos.

Operaciones comunes con sets

Unión (`union`): Combina todos los elementos únicos de dos conjuntos.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
combined = set1.union(set2)  # {1, 2, 3, 4, 5}
```

-

Intersección (`intersection`): Devuelve los elementos comunes a ambos conjuntos.

```
common = set1.intersection(set2)  # {3}
```

-

Eliminar duplicados de una lista:

```
numbers = [1, 2, 2, 3, 3, 4]
unique_numbers = set(numbers)  # {1, 2, 3, 4}
```

-

Aplicaciones prácticas de los sets

1. Gestión de clientes:

Almacenar correos electrónicos únicos para evitar envíos duplicados.

2. Inventarios y logística:

Garantizar que cada producto se registre una sola vez.

3. Redes sociales:

Filtrar hashtags o palabras clave duplicadas y analizar tendencias.

4. Análisis de datos:

Extraer respuestas únicas de encuestas o encontrar intersecciones entre diferentes conjuntos de datos (clientes, habilidades, productos, etc.).

Pregunta

Estás intentando dar con un ejemplo en el que el uso de un conjunto Python sería la estructura de datos MÁS apropiada y eficiente. ¿Cuál de los siguientes escenarios lo ilustra mejor? Elige la mejor respuesta.

- ☒ Recopilación de una lista única de direcciones de correo electrónico a partir de una base de datos para evitar el envío de correos electrónicos promocionales duplicados.
- ☐ Asignación de ID de productos a sus precios correspondientes en una tienda en línea.
- ☐ Mantener una secuencia de acciones del usuario en un videojuego, donde es necesario preservar el orden de las acciones.
- ☐ Almacenar una lista de nombres de alumnos en el orden en que se matricularon en una clase.

✔ Correcto

Correcto. Los conjuntos son ideales para garantizar la unicidad, que es esencial para evitar la duplicación de correos electrónicos.

Organizar datos con estructuras de datos

Las **estructuras de datos** son herramientas esenciales para organizar y acceder a información de manera eficiente. Elegir la estructura adecuada depende del tipo de datos y de cómo planeas usarlos.

Tipos principales y sus usos

1. Listas (**list**)

- Colecciones ordenadas.
- Ideales cuando el **orden importa**, como pasos de una receta, niveles de un juego o listas de tareas.

Ejemplo:

```
levels = ["nivel1", "nivel2", "nivel3"]
```

○

2. Diccionarios (**dict**)

- Colecciones de **pares clave-valor**.
- Útiles para etiquetar elementos y acceder a ellos rápidamente, como almacenar perfiles de usuario o configuraciones de aplicaciones.

Ejemplo:

```
user_profiles = {"user123": {"nombre": "Alice", "nivel": 5}}
```

○

3. Conjuntos (**set**)

- Colecciones de **elementos únicos y sin orden**.
- Perfectos para eliminar duplicados y rastrear elementos únicos, como visitantes de un sitio web, clases de personajes en un juego o palabras únicas en un texto.

Ejemplo:

```
unique_boosters = {"escudo", "poción", "bomba"}
```

○

Beneficios

- Usar la estructura correcta **acelera la aplicación** y facilita el manejo de los datos.
- Permite **almacenar y acceder a información de manera más intuitiva** y eficiente.
- Facilita la limpieza de datos y la gestión de elementos únicos.

Estructuras de datos: organizar información en Python

Las **estructuras de datos** son contenedores especializados que permiten almacenar y organizar información de manera eficiente. Tres estructuras fundamentales en Python son **listas**, **diccionarios** y **conjuntos**.

1. Listas (**list**) – Colecciones ordenadas y mutables

- Similares a una lista de tareas: cada elemento tiene un **orden** y un **índice**.
- **Mutables**: puedes añadir, eliminar o reorganizar elementos.
- **Indexables**: se accede a los elementos mediante su índice (empieza en 0).

Operaciones comunes:

```
my_list = ["tarea1", "tarea2", "tarea3"]
```

```
my_list.append("tarea4")      # Añade al final
my_list.insert(1, "tarea1.5") # Inserta en posición específica
my_list.pop()                 # Elimina y devuelve el último
                               # elemento. El método pop() puede eliminar y devolver el último
                               # elemento de la lista, o puede especificar el índice del invitado a
                               # eliminar.
my_list.remove("tarea2")      # Elimina la primera aparición de un
                               # elemento
del my_list[0]                # Elimina un elemento por índice

list.clear()                  # Elimina todos los elementos de la
                               # lista
```

- Ideal para **colecciones ordenadas** donde el **orden importa**.

2. Diccionarios (**dict**) – Pares clave-valor

- Similares a **cajas etiquetadas**: cada clave es única y apunta a un valor.
- **Ordenados (Python 3.7+)**: mantienen el orden de inserción.
- **Mutables**: puedes añadir, actualizar o eliminar pares clave-valor.
- Permiten **búsquedas rápidas** usando la clave.

Ejemplo práctico: almacenar contactos:

```
# Crear un diccionario
contacts = {"Alice": "555-1234", "Bob": "555-5678", "Carol": "555-9012"}

# Recuperar el número de Bob
bobs_phone = contacts["Bob"]
print(bobs_phone) # Output: 555-5678

# Añadir un nuevo contacto
contacts["David"] = "555-4321"

# Actualizar el número de Carol
contacts["Carol"] = "555-2468"

# Eliminar el contacto de Alice
del contacts["Alice"]

# Mostrar contactos actualizados
print(contacts)
# Output: {'Bob': '555-5678', 'Carol': '555-2468', 'David': '555-4321'}
```

- Útil para **almacenar información asociada a claves únicas**, como contactos, configuraciones o bases de datos simples.

Diccionarios (**dict**) – Almacenamiento clave-valor

- **Descripción:** Colecciones de pares clave-valor. Cada clave es única y apunta a un valor.
- **Mutable y ordenados (Python 3.7+):** Puedes añadir, actualizar o eliminar pares clave-valor.
- **Aplicaciones reales:**
 - Libretas de direcciones: almacenar nombres y teléfonos.
 - Catálogos de productos: ID de producto → detalles y stock.

- Configuración de usuario: preferencias de idioma, tema, etc.
- Diccionarios de traducción: palabra de un idioma → equivalente en otro.

Conjuntos (**set**) – Colecciones únicas

- **Descripción:** Colección de elementos únicos; ideal para eliminar duplicados y verificar pertenencia.
- **Características principales:**
 - **Desordenados:** el orden no se conserva.
 - **Mutables:** se pueden añadir o eliminar elementos.
 - **Únicos:** cada elemento solo puede aparecer una vez.

Operaciones comunes

`add(item)`: Añade `item` al conjunto sólo si no está ya presente.

`remove(item)`: Elimina `item` del conjunto si ya existe.

`union(other_set)`: Combina dos conjuntos para crear un nuevo conjunto que contenga todos los elementos únicos de ambos.

`intersection(other_set)`: Crea un nuevo conjunto que contiene sólo los elementos comunes a ambos conjuntos.

`difference(other_set)`: Crea un nuevo conjunto que contiene elementos que están en el primer conjunto pero no en el segundo.

`issubset(other_set)`: Comprueba si todos los elementos del primer conjunto están también en el segundo.

`issuperset(other_set)`: Comprueba si todos los elementos del segundo conjunto están también en el primero.

Operaciones comunes y ejemplo de código:

```
# Crear un conjunto de lenguajes de programación favoritos
languages = {"Python", "JavaScript", "Java"}

# Añadir un nuevo lenguaje
languages.add("C++")

# Intentar añadir "Python" nuevamente (no se añade porque ya existe)
languages.add("Python")
print(languages)
# Output: {'Python', 'C++', 'JavaScript', 'Java'} (el orden puede variar)

# Eliminar un elemento
languages.remove("Java")

# Crear otro conjunto
web_languages = {"JavaScript", "HTML", "CSS"}

# Encontrar elementos comunes entre los conjuntos
common_languages = languages.intersection(web_languages)
print(common_languages) # Output: {'JavaScript'}
```

- **Aplicaciones reales:**

- Eliminar duplicados en listas o bases de datos.
- Comprobar si un elemento existe en una colección grande.
- Encontrar intereses comunes entre usuarios.
- Determinar palabras únicas en un documento.

Elegir la estructura adecuada

Necesidad

**Estructura
recomendada**

Ejemplo de uso

Colección ordenada con duplicados	Lista (list)	Lista de tareas o pasos de una receta
Asociar claves únicas a valores	Diccionario (dict)	Número de teléfono de un alumno por nombre
Garantizar unicidad y operaciones de conjunto	Conjunto (set)	Colección de sellos, intereses únicos de usuarios

- Comprender **puntos fuertes y limitaciones** de cada estructura permite **organizar datos eficazmente** y escribir código más limpio y eficiente.

Orden VS Desorden

Estructuras de datos ordenadas

- **Definición:** Mantienen los elementos en el orden en que se agregaron o insertaron. La secuencia importa.
- **Ejemplos:**
 - **Listas (**list**):** Colecciones flexibles que permiten acceder por índice, modificar, añadir, eliminar, ordenar o invertir elementos.
 - **Tuplas (**tuple**):** Colecciones inmutables que representan conjuntos de datos fijos, como registros de bases de datos.
- **Ventajas:**
 - Acceso directo por índice.
 - Permite procesar datos paso a paso en secuencias.
 - Mantiene relaciones y dependencias entre elementos.
- **Aplicaciones prácticas:**
 - Listas de reproducción de música (orden de canciones).

- Gestión de tareas y proyectos (prioridad y fechas de vencimiento).
- Colas para solicitudes de atención al cliente.
- Análisis de series temporales, como patrones climáticos.

Estructuras de datos desordenadas

- **Definición:** No garantizan el orden de los elementos; se centran en la pertenencia y relaciones entre los elementos más que en la posición.
- **Ejemplos:**
 - **Conjuntos (set):** Colecciones de elementos únicos, sin duplicados.
- **Ventajas:**
 - Comprobación rápida de si un elemento está presente (prueba de membresía).
 - Operaciones de conjuntos eficientes: unión, intersección, diferencia, etc.
- **Aplicaciones prácticas:**
 - Verificación de credenciales de usuarios.
 - Eliminación automática de duplicados en datos.
 - Identificación de elementos comunes entre colecciones.

Conclusión

- La **elección de estructura de datos** depende de tus necesidades:
 - **Orden y secuencia importante** → usar estructuras ordenadas (listas, tuplas).
 - **Unicidad y pertenencia rápida** → usar estructuras desordenadas (conjuntos).

- Comprender estas diferencias permite **acceder y manipular datos de manera eficiente**, optimizando el rendimiento y la lógica del programa.

Pregunta

Estás trabajando en un proyecto de Python que requiere que proceses los tickets de atención al cliente en el orden exacto en que fueron recibidos. ¿Qué tipo de estructura de datos sería la más adecuada para almacenar y gestionar estos tickets? Selecciona la mejor respuesta.

- ☐ Un diccionario
- ☐ Un conjunto
- ☒ Una estructura de los datos ordenada
- ☐ Estructura de los datos desordenada

✔ **Correcto**
Correcto Las estructuras de datos ordenadas mantienen la secuencia de los elementos, lo que las hace ideales para manejar tareas que requieren un procesamiento en un orden específico.

Cuestionario

1. Estás construyendo un programa para seguir la lista de la compra de un usuario. ¿Qué estructura de los datos Python sería la más adecuada para almacenar la lista de artículos que quieren comprar, donde el orden de los artículos importa? Elige la mejor respuesta.

- ☒ Lista
- ☐ Diccionario
- ☐ Tupla
- ☐ Fije

✔ **Correcto**
Correcto Las listas están diseñadas para almacenar colecciones ordenadas de elementos, por lo que son ideales para representar una lista de la compra en la que el orden de los elementos puede ser importante.

2. Necesita almacenar una colección de números de identificación de estudiantes, donde cada número debe ser único. ¿Qué estructura de los datos de Python sería la más adecuada para esta tarea? Elige la mejor respuesta.

- ☐ Diccionario
- ☐ Lista
- ☐ Tupla
- ☒ Fije

✔ **Correcto**
¡Correcto! Los conjuntos eliminan automáticamente los duplicados, por lo que son ideales para almacenar elementos únicos como los números de identificación de los estudiantes.

Conjuntos

3. Estás escribiendo un programa para almacenar las coordenadas de la ubicación de un usuario (latitud y longitud). ¿Qué estructura de los datos de Python sería la opción más adecuada, teniendo en cuenta que estas coordenadas no deben modificarse una vez establecidas? Elige la mejor respuesta.

- ☐ Diccionario
- ☐ Fije
- ☐ Lista
- ☒ Tupla

✓ **Correcto**

Correcto Las tuplas son inmutables, lo que garantiza que las coordenadas permanezcan inalteradas una vez asignadas, proporcionando integridad a los datos.

4. Estás desarrollando un programa para analizar los precios históricos de las acciones a lo largo del tiempo. ¿Qué tipo de estructura de los datos sería la más adecuada para almacenar estos precios, teniendo en cuenta la importancia de mantener su orden cronológico? Elige la mejor respuesta.

- ☐ Inmutable
- ☐ Mutable
- ☐ Desordenado
- ☒ Encargado *ordenado*

✓ **Correcto**

Correcto Las estructuras de datos ordenadas, como las listas, conservan la secuencia en la que se añaden los elementos, lo que las hace ideales para almacenar datos de series temporales en las que el orden representa la línea de tiempo.

5. Estás diseñando un juego en el que las puntuaciones más altas de los jugadores deben almacenarse y mostrarse en orden descendente. ¿Qué par de estructuras de datos sería el más adecuado para esta tarea? Elige la mejor respuesta.

- ☐ Diccionario para partituras, Lista para visualización
- ☐ Set para partituras, Diccionario para visualización
- ☒ Lista de partituras, Lista de visualización
- ☐ Tuple para las puntuaciones, Set para la visualización

✓ **Correcto**

Correcto Las listas mantienen el orden y pueden clasificarse fácilmente, por lo que son perfectas para almacenar y mostrar puntuaciones altas en orden descendente.

Manipulación de datos con Python (vídeo diferente muy orientado a los tres pilares de los datos, que incluye también mucho código y manejo de DataFrames con pandas)

DE HECHO EL VÍDEO ES LITERALMENTE EL EJEMPLO DE IRIS DE TAC

La **manipulación de datos** en Python consiste en transformar datos crudos en información valiosa para el análisis y la toma de decisiones. Es comparable a pulir una joya para revelar su valor.

Bibliotecas principales:

1. **pandas**: Permite trabajar con datos tabulares mediante **DataFrames**, similares a hojas de cálculo. Facilita filtrar, ordenar, agrupar, pivotar y fusionar datos. Funciones importantes:
 - `df.head()`, `df.info()`, `df.describe()` → exploración de datos.
 - `df.dropna()`, `df.fillna()` → manejo de valores faltantes.
 - `df.drop_duplicates()`, `df.reset_index()` → limpieza y reestructuración de datos.
 - `df.groupby().sum()/mean()/count()` → agregaciones y resúmenes.
 - `pd.merge()` y `pd.concat()` → unir y combinar DataFrames.
 - `df.sort_values()` → ordenar datos.
 - `df.apply()` → aplicar funciones personalizadas.

- Métodos de cadenas y `replace()` → manipulación y limpieza de datos de texto.
 - Funciones de fechas y horas para análisis temporal.
2. **NumPy**: Proporciona **matrices de alto rendimiento** y operaciones numéricas eficientes. pandas se basa en NumPy para cálculos internos.
 3. **Matplotlib.pyplot**: Biblioteca de **visualización de datos**, ideal para gráficos y tendencias basadas en datos de pandas o NumPy.

Procesos clave en manipulación de datos:

- Limpieza de datos: eliminar valores faltantes, duplicados y caracteres indeseados.
- Transformación: pivotar, apilar/desapilar y reestructurar DataFrames.
- Agregación y resumen: agrupar por categorías y aplicar sumas, medias o conteos.
- Combinación: unir y concatenar diferentes DataFrames para análisis completo.
- Filtrado y selección: extraer filas o columnas según criterios específicos.
- Personalización: aplicar funciones personalizadas a columnas o filas enteras.
- Texto y fechas: limpiar y manipular cadenas, gestionar datos temporales y realizar cálculos por frecuencia.

Objetivo final: convertir datos complejos en información organizada, lista para análisis y visualización, optimizando la toma de decisiones.

Diccionarios en profundidad

Los **diccionarios** en Python son herramientas potentes para organizar y gestionar datos de manera eficiente. Más allá de las operaciones básicas, existen **técnicas avanzadas** que permiten manejar estructuras de datos complejas y escribir código más limpio y comprensible.

Principales técnicas avanzadas:

1. **Diccionarios anidados:**
 - Permiten almacenar información jerárquica dentro de un diccionario.

- Ejemplo: un sistema de gestión de proyectos donde un proyecto contiene un diccionario con sus tareas, responsables, fechas y estados.
- Facilita visualizar relaciones complejas y manejar múltiples niveles de información.

2. Iteración sobre claves y valores:

- Permite recorrer sistemáticamente cada elemento de un diccionario.
- Ejemplo: calcular ventas totales por producto y región usando diccionarios anidados.

3. Método `get()`:

- Permite acceder a un valor de manera segura sin provocar errores si la clave no existe.
- Puede devolver un valor predeterminado, útil para gestionar datos faltantes.
- Ejemplo: obtener preferencias de usuario con un valor por defecto si la clave no está definida.

4. Comprensión de diccionarios:

- Crear nuevos diccionarios a partir de existentes aplicando transformaciones o filtrando elementos.

5. Fusión de diccionarios:

- Combina varios diccionarios en uno solo, consolidando datos de distintas fuentes o actualizando información existente.

6. Clasificación de diccionarios:

- Organiza los elementos según claves o valores, facilitando la presentación de datos.

Beneficios:

- Mantener datos jerárquicos organizados.
- Agilizar la manipulación de datos complejos.
- Facilitar la lectura y comprensión del código.

- Brindar flexibilidad para filtrar, transformar, fusionar y ordenar información.

Conclusión:

Dominar estas técnicas avanzadas convierte a un programador en más eficiente y capaz de manejar desafíos de datos del mundo real con confianza.

Pregunta

Estás trabajando en un proyecto de Python que implica la gestión de datos complejos y jerárquicos sobre los proyectos de una empresa. Cada proyecto tiene múltiples tareas, y cada tarea tiene su propio conjunto de detalles (asignatario, fecha límite, estado, etc.). ¿Qué técnica de diccionario avanzado sería la MÁS adecuada para organizar y acceder a estos datos de forma eficiente?

- ☒ Diccionarios anidados
- ☐ Comprensiones de diccionario
- ☐ Iteración sobre claves y valores
- ☐ El método `get()`

✓ Correcto

Esta es la respuesta correcta. Los diccionarios anidados permiten crear varios niveles de pares clave-valor dentro de un mismo diccionario, lo que los hace ideales para representar la relación jerárquica entre los proyectos y sus tareas.

Aplicaciones en el mundo real y cuando utilizar cada estructura de datos (Lo enseña con código en el vídeo)

En **Python**, elegir la estructura de datos correcta es clave para organizar, acceder y manipular información de manera eficiente. Diferentes estructuras se destacan en distintos escenarios según la necesidad de rapidez, eficiencia en inserción/eliminación o representación de relaciones complejas.

Principales estructuras de datos y sus aplicaciones prácticas:

1. Arrays (matrices):

- Almacenan elementos en bloques contiguos de memoria.
- Útiles para acceso rápido y operaciones numéricas.

2. Listas enlazadas:

- Elementos conectados mediante punteros.

- Flexibles para insertar y eliminar elementos sin reorganizar toda la colección.

3. Pilas (Stacks, LIFO):

- Último en entrar, primero en salir.
- Ejemplo: historial de navegación; el botón “Atrás” funciona mediante pop de la pila.

4. Colas (Queues, FIFO):

- Primero en entrar, primero en salir.
- Ejemplo: listas de tareas pendientes; se pueden usar estructuras como **deque** para añadir o quitar tareas desde ambos extremos. Cola de la impresora.

5. Conjuntos (Sets):

- Colecciones únicas y desordenadas de elementos.
- Ejemplo: comprobar rápidamente si una canción ya está en tu lista de reproducción.

6. Diccionarios y diccionarios anidados:

- Almacenan pares clave-valor, ideales para representar relaciones jerárquicas.
- Ejemplo: red social donde cada usuario tiene una lista de amigos; catálogo de contactos con detalles anidados (correo, teléfono).

7. Listas anidadas:

- Útiles para representar datos tabulares o imágenes.
- Ejemplo: cada fila de píxeles es una lista, y cada píxel es una tupla RGB, permitiendo manipulación de imágenes digitales.

8. Árboles:

- Estructuras jerárquicas con nodo raíz y ramas.
- Ejemplo: sistema de archivos; carpetas y subcarpetas se representan como nodos y ramas del árbol.

9. Gráficos (Graphs):

- Representan relaciones entre elementos mediante nodos y aristas.
- Útiles para modelar redes sociales, conexiones entre usuarios, rutas o dependencias.

Consideraciones para elegir la estructura adecuada:

- ¿Los elementos están ordenados o son únicos?
- ¿Necesitas relaciones complejas entre elementos?
- ¿Insertar, eliminar o buscar elementos de forma frecuente?
- ¿Qué tan importante es la velocidad y la eficiencia de memoria?

Conclusión:

No existe una estructura perfecta para todas las situaciones. La elección depende de los requisitos del proyecto. Analizar los datos, entender las operaciones necesarias y conocer las fortalezas de cada estructura garantiza eficiencia y organización en tus aplicaciones Python.

Pregunta

Escenario: Usted está creando un programa para almacenar una lista de tareas que necesitan ser completadas en el orden en que fueron agregadas. También desea poder comprobar rápidamente si una tarea específica ya está en la lista.

Vástago de la pregunta: ¿Qué combinación de estructuras de datos sería la más adecuada para este programa? Seleccione la mejor respuesta.

- ☐ Una "lista" para almacenar las tareas y un "diccionario" para encontrar rápidamente tareas específicas.
- ☒ Sólo una "lista" tanto para almacenar las tareas como para buscar tareas específicas.
- ☐ Un "conjunto" para almacenar las tareas y una "lista" para llevar la cuenta del orden.
- ☐ Un "árbol" para organizar las tareas jerárquicamente y una "cola" para procesarlas por orden.

✔ **Correcto**

Correcto Una "lista" puede tanto almacenar tareas en orden como permitir comprobar si una tarea existe, lo que la hace adecuada para este programa.

Ejercicios de estructura de los datos: La práctica hace al maestro

Paso 1: Cargar y explorar los datos

Comenzará utilizando la biblioteca `pandas` para cargar los datos de sus productos desde un Archivo CSV (`inventory.csv`). Este archivo sirve como columna vertebral de nuestro sistema de inventario, almacenando información vital sobre cada producto. A continuación, extraeremos los nombres de los productos y los almacenaremos en una lista para facilitar su manipulación y exploración.

- El código que importa `pandas` ya se ha proporcionado para usted.
- Cargue los datos del inventario desde el archivo `inventory.csv` utilizando `pd.read_csv('inventory.csv')`.
- Ejecute la celda, que extraerá los nombres de los productos del DataFrame y los almacenará en una lista llamada `inventory` e imprimirá su salida.

Paso 2: Eliminar un artículo

Los productos van y vienen y necesitará mantener la lista de inventario actualizada. En este paso, eliminará un artículo específico, "Cheesy Chompers", de la lista de inventario, asegurándose de que sus registros reflejen con precisión los productos disponibles.

- Utilice el método `remove()` para eliminar la primera aparición de "Cheesy Chompers" de la lista `inventory`.
- Ejecute la celda para imprimir la lista `inventory` actualizada y ver los cambios.

1. ¿Cuál de los siguientes métodos se utiliza para eliminar un elemento específico de una lista en Python?
Seleccione la mejor respuesta.

1 / 1 punto

- ☐ `delete()`
- ☐ `pop()`
- ☐ `del`
- ☒ `remove()`

✔ Correcto

Correcto El método `remove()` elimina de la lista la primera aparición de un valor especificado.

2. Paso 3: Añadir nuevos artículos

Además de eliminar artículos, a veces también necesitará añadirlos a su lista de inventario. En este paso, convertirá una representación de cadena de estos nuevos artículos en una lista y los añadirá sin problemas a su inventario existente.

- El código que convierte la cadena separada por comas `new_items_str` en una lista de elementos individuales utilizando el método `split()` ya se ha proporcionado para usted.
- Añada todos los nuevos artículos de la lista `new_items` al final de la lista `inventory` existente utilizando el método `extend()`.
- Ejecute la celda para imprimir la lista `inventory` actualizada y ver los nuevos elementos añadidos.

¿Qué método se utiliza en Python para añadir varios elementos nuevos al final de una lista existente?
Selecciona la mejor respuesta.

- ☐ `add()`
- ☒ `extend()`
- ☐ `insert()`
- ☐ `append()`

✔ **Correcto**

Correcto El método `extend()` añade todos los elementos de otro iterable al final de la lista.

3. Paso 4: Trabajar con tuplas

En el mundo de la gestión de datos, la inmutabilidad desempeña un papel crucial en la preservación de la integridad de los datos. En este paso, explorarás el concepto de tuplas, que son colecciones ordenadas e inmutables de elementos que son ideales para almacenar información sensible como IDs de productos.

- El código para crear algunos IDs de productos como tuplas e imprimirlos se ha proporcionado para usted.
- Ejecute la celda y observe la salida.
- Descomente la última línea de código eliminando "#".
- Ejecute la celda por segunda vez y observe la `TypeError` que destaca su inmutabilidad.

¿Cuál de las siguientes estructuras de datos es la más adecuada para almacenar ID de productos que no deben modificarse después de su creación? Seleccione la mejor respuesta.

- ☐ Diccionario
- ☐ Lista
- ☒ Tupla
- ☐ Fije

✔ **Correcto**

Correcto Las tuplas son inmutables, por lo que son ideales para almacenar datos que no deben modificarse.

4. Paso 5: Cargar los datos de inventario en un diccionario

A medida que su inventario crezca, necesitará una forma más estructurada y eficiente de gestionar la información de los productos. En este paso, cargará sus datos de inventario desde un Archivo CSV directamente a un diccionario, donde los nombres de los productos actúan como claves y sus correspondientes niveles de existencias como valores.

- Ejecute la celda con el código proporcionado, que carga los datos de inventario desde el archivo 'inventory.csv', convierte el DataFrame directamente en un diccionario e imprime el diccionario inicial `inventory_dict`.
- Observe el resultado.

¿Cuál es la principal ventaja de utilizar un diccionario en lugar de una lista para la gestión del inventario? Seleccione la mejor respuesta.

- ☐ Los diccionarios ordenan automáticamente los artículos del inventario por orden alfabético.
- ☐ Los diccionarios permiten una iteración más rápida sobre todos los artículos del inventario.
- ☐ Los diccionarios son inmutables, lo que impide cambios accidentales en los datos del inventario.
- ☒ Los diccionarios permiten consultar y modificar eficazmente los niveles de existencias utilizando los nombres de los productos como claves.

✓ **Correcto**

Correcto Esta estructura clave-valor permite acceder y actualizar rápidamente la información del inventario.

5. Paso 6: Actualizar el diccionario de inventario

La naturaleza dinámica de la gestión del inventario requiere que actualice constantemente sus registros. En este paso, agregará nuevos productos a su diccionario de inventario y modificará los niveles de existencias de los existentes, asegurándose de que sus datos sigan siendo precisos y reflejen los últimos cambios.

- Añada un nuevo artículo, "Snacks para cachorros", al diccionario `inventory_dict` con un nivel de existencias de 40.
- Actualice el nivel de existencias de "Cheesy Chompers" en el diccionario `inventory_dict` a 20.
- Ejecute la celda para imprimir el diccionario `inventory_dict` actualizado y ver los cambios.

Paso 7: Eliminar un artículo del diccionario

En este paso, eliminará artículos específicos de su diccionario de inventario utilizando la palabra clave `del`, para mantener sus registros limpios y despejados.

- Utilice la palabra clave `del` para eliminar la entrada "Galletas de mantequilla de cacahuete" del diccionario `inventory_dict`.
- Ejecute la celda para imprimir el diccionario `inventory_dict` actualizado y ver los cambios.

¿Cuál de las siguientes afirmaciones describe con exactitud las características clave que distinguen a las listas, tuplas y diccionarios en Python? Selecciona la mejor respuesta.

- ☐ Las listas son inmutables y ordenadas, las tuplas son inmutables y desordenadas, y los diccionarios son mutables y desordenados.
- ☐ Las listas son mutables, mientras que los diccionarios y las tuplas son inmutables. Las tres estructuras de datos son colecciones ordenadas.
- ☒ Las listas y los diccionarios son mutables, mientras que las tuplas son inmutables. Los tres están ordenados.
- ☐ Las tres estructuras de datos son mutables y están ordenadas. Las listas y las tuplas utilizan índices numéricos para acceder a los elementos, mientras que los diccionarios utilizan claves.

✓ **Correcto**

Correcto Esta afirmación resume con precisión las principales diferencias entre estas estructuras de datos

Cuestionario

1. ¿Cuál de las siguientes afirmaciones describe correctamente la forma de eliminar elementos de una lista en Python? Seleccione todas las que correspondan.

☒ El método `remove()` elimina la primera aparición de un valor.

☒ **Correcto**

Correcto El método `remove()` elimina el primer valor coincidente, no por índice.

☒ El método `clear()` elimina todos los elementos de la lista.

☒ **Correcto**

Correcto El método `clear()` vacía la lista, eliminando todos sus elementos.

☐ El método `delete()` puede eliminar un elemento por índice.

☒ El método `pop()` puede eliminar un elemento por índice.

☒ **Correcto**

Correcto El método `pop()` elimina y devuelve el elemento en el índice dado.

2. Estás diseñando un sistema para realizar un seguimiento de las acciones del usuario en un programa de edición de fotos. Necesitas implementar una función de "deshacer" que revierta primero la acción más reciente. ¿Qué estructura de datos sería la más adecuada para almacenar estas acciones y garantizar que la función "deshacer" funcione correctamente? Selecciona la mejor respuesta.

☒ Pila

☐ Array

☐ Lista enlazada

☐ Cola

☒ **Correcto**

Correcto Una pila encarna perfectamente el principio LIFO (Last In, First Out). La acción más reciente añadida a la pila será la primera eliminada cuando el usuario realice un "deshacer".

3. ¿En qué escenario del mundo real se utilizaría más eficazmente una estructura de datos en cola? Seleccione la mejor respuesta.

- ☐ Almacenamiento de datos jerárquicos, como las rutas del sistema de archivos.
- ☐ Encontrar el camino más corto en un sistema de navegación.
- ☒ Gestión de trabajos de impresión.
- ☐ Implementar una funcionalidad de deshacer en un editor de texto.

✔ **Correcto**

Correcto Una cola garantiza que los trabajos de impresión se gestionan en el orden en que se reciben.

4. ¿Cuál de los siguientes métodos se utiliza para añadir un elemento al final de una lista en Python? Selecciona la mejor respuesta.

- ☐ insertar()
- ☐ añadir()
- ☒ append()
- ☐ ampliar()

✔ **Correcto**

Correcto El método append() se utiliza para añadir un elemento al final de una lista.

5. Estás gestionando una lista de invitados para un evento utilizando una lista de Python. De repente, te das cuenta de que uno de los invitados ya no puede asistir. Necesitas eliminar su nombre de la lista. ¿Cuál de los siguientes métodos de Python sería apropiado para eliminar el nombre del invitado? Seleccione todos los que correspondan.

- ☐ suprimir()
- ☐ insertar()
- ☒ pop()

✔ **Correcto**

Correcto El método pop() puede eliminar y devolver el último elemento de la lista, o puede especificar el índice del invitado a eliminar.

☒ eliminar()

✔ **Correcto**

Correcto El método remove() encontrará y eliminará de la lista la primera aparición del nombre del invitado.

6. ¿Cuál de las siguientes afirmaciones sobre el método `get()` en los diccionarios de Python es cierta? Selecciona la mejor respuesta.

- ☐ El método `get()` sólo puede utilizarse en diccionarios con claves de cadena.
- ☐ El método `get()` generará un `KeyError` si no se encuentra la clave especificada.
- ☒ El método `get()` permite especificar un valor por defecto si no se encuentra la clave.
- ☒ El método `get()` modifica el diccionario añadiendo el valor por defecto si no se encuentra la clave. *NO!*

✓ **Correcto**

Correcto Este es uno de los usos principales del método `get()`.

Ordenación de listas en Python

La **clasificación en Python** consiste en reorganizar los elementos de una lista en un orden específico (numérico, alfabético o definido por criterios personalizados). Esto facilita la **organización, el análisis y la presentación de datos**, permitiendo detectar tendencias, patrones y valores atípicos.

Existen varios **algoritmos de clasificación**:

- **Burbuja**: compara pares de elementos adyacentes e intercambia si están en orden incorrecto.
- **Selección**: busca el elemento más pequeño o grande y lo coloca en la posición correcta.
- **Rápida (quicksort)**: divide la lista en grupos según un pivote y ordena cada parte de manera recursiva.

Además, Python permite **clasificación personalizada**, útil para ordenar por atributos (ej. productos por precio y valoración).

La clasificación es importante porque:

- Hace más eficiente la **búsqueda** (ej. con búsqueda binaria).
- Mejora la **claridad y comprensión** de la información.
- Facilita el **análisis de datos** al revelar patrones.
- Optimiza la **experiencia del usuario** en aplicaciones como e-commerce o gestión de datos.

En resumen: clasificar listas en Python no solo organiza datos, también los vuelve más útiles y accesibles para la toma de decisiones.

Pregunta

Estás trabajando en una herramienta de gestión de proyectos y necesitas mostrar una lista de las próximas tareas al usuario. ¿Cuál de las siguientes situaciones pone MEJOR de manifiesto la importancia de la ordenación en este contexto? Seleccione la mejor respuesta.

- ☐ Desea permitir a los usuarios personalizar el orden en que se muestran las tareas, en función de sus preferencias individuales.
- ☐ Debe asegurarse de que se muestren todas las tareas, independientemente de sus fechas de vencimiento o niveles de prioridad.
- ☐ Es necesario presentar las tareas en un orden aleatorio para mantener al usuario interesado y sorprendido.
- ☒ Desea priorizar las tareas en función de sus fechas de vencimiento, asegurándose de que las tareas más urgentes aparecen al principio de la lista.

✔ **Correcto**

Correcto Clasificar las tareas por fecha de vencimiento es una función común y esencial en las herramientas de gestión de tareas, que ayuda a los usuarios a centrarse en lo que requiere atención inmediata.

Búsqueda de listas en Python (Tiene código en el vídeo)

Las **listas en Python** son colecciones ordenadas de elementos a las que se accede por índice. Buscar dentro de ellas consiste en localizar un elemento específico, y para ello existen distintos **algoritmos de búsqueda**.

Búsqueda lineal

- Recorre cada elemento de la lista secuencialmente hasta encontrar el objetivo o llegar al final.
- Funciona con listas pequeñas o no ordenadas.
- Es fácil de implementar pero poco eficiente en listas grandes.

Búsqueda binaria

- Requiere que la lista esté **ordenada**.
- Divide repetidamente la lista a la mitad, comparando el elemento central con el objetivo.
- Muy rápida y eficiente para listas grandes.
- Usada en aplicaciones como análisis de datos, motores de búsqueda o sistemas de recomendación.

Elección del algoritmo

- **Lista pequeña o desordenada** → búsqueda lineal.
- **Lista grande y ordenada** → búsqueda binaria.

Otras herramientas en Python

- Operador **in**: comprueba si un elemento está en una lista.
- **index()**: devuelve la posición de la primera aparición de un elemento.
- **Comprensiones de listas**: crean nuevas listas según criterios de búsqueda.
- Módulo **bisect**: gestiona listas ordenadas e inserciones rápidas manteniendo el orden.

Conclusión

Buscar en listas es una **habilidad esencial** en Python, con aplicaciones reales como bases de datos, motores de búsqueda y sistemas de recomendación. Dominar la búsqueda lineal y binaria, junto con las herramientas adicionales, mejora la eficiencia y comprensión del manejo de datos.

ACTIVIDAD: Encontrar lo que necesitas

Escenario

Gestiona el catálogo de una biblioteca digital en expansión. A medida que la colección se amplía, resulta cada vez más difícil mantener los libros organizados y garantizar que los usuarios puedan encontrar rápidamente lo que buscan. La clasificación y la búsqueda manuales llevarían mucho tiempo y serían ineficaces. Aquí es donde entra en juego el poder de los algoritmos de clasificación y búsqueda. Mediante la aplicación de estas técnicas, puede crear un catálogo fácil de usar que permite una fácil navegación y una rápida recuperación de cualquier libro, no importa lo grande que sea la biblioteca.

Objetivo

El objetivo de esta actividad es dotarle de un conocimiento práctico de los algoritmos de ordenación y búsqueda, herramientas esenciales para gestionar y acceder a los datos de forma eficaz. Explorará cómo la ordenación organiza los datos en un orden lógico, facilitando la navegación y la localización de elementos específicos. También aprenderá cómo los algoritmos de búsqueda, como la búsqueda lineal y binaria, permiten recuperar información de forma rápida y precisa a partir de grandes conjuntos de datos.

Al final de esta actividad, no sólo comprenderá el funcionamiento interno de estos algoritmos, sino que también apreciará su importancia en aplicaciones del mundo real, desde catálogos de bibliotecas hasta motores de búsqueda. Adquirirá experiencia práctica en la implementación de estos algoritmos en Python, lo que le permitirá enfrentarse con confianza a los retos de la organización y recuperación de datos.

Abre la carpeta, que contiene los siguientes archivos

- un archivo de Notebook de Jupyter llamado "PYTC1M4P02_Activity - Finding what you need.ipynb"
- un conjunto de datos de valores separados por comas (CSV) llamado "book_catalog_10.csv"
- un conjunto de datos de valores separados por comas (CSV) denominado "big_book_catalog.csv"

Tenga en cuenta que se le calificará utilizando preguntas de opción múltiple, no su archivo Jupyter. Si necesita volver a empezar, puede descargar de nuevo este archivo. El cuaderno de soluciones se proporciona al final de la tarea como referencia.

Paso 1: Cargar y explorar los datos

Empiece cargando el catálogo de libros desde un Archivo CSV. Este archivo actúa como un catálogo de tarjetas de biblioteca digital, almacenando información sobre cada libro. Utilizaremos la biblioteca `pandas` para leer y estructurar sin esfuerzo estos datos dentro de su entorno Python.

- Se ha proporcionado un Archivo CSV ('book_catalog_10.csv') que contiene los datos iniciales de los libros.
- El código para importar pandas, cargar el CSV, y crear un DataFrame (`book_catalog_df`) ya está en su lugar.
- Ejecute la primera celda para ejecutar este código.
- La siguiente celda contiene código para convertir el DataFrame en una lista de diccionarios (`book_catalog`) para facilitar su manejo.
- Ejecute esta celda y observe la salida impresa, mostrando la estructura de los datos de su libro.

Paso 2: Ordenar el catálogo de libros

Ahora, ¡organice su biblioteca digital! Al igual que ordenaría los libros en una estantería, ordenará nuestros datos de libros alfabéticamente por título. De este modo, será mucho más fácil navegar y encontrar libros concretos más adelante.

- Las funciones de ayuda (`get_title` y `sort_catalog_by_title`) se han proporcionado para ayudar con el proceso de clasificación.
- Revise estas funciones para entender cómo extraen los títulos y realizan la ordenación.
- Ejecute la celda para ejecutar la clasificación e imprima el catálogo clasificado.
- Observe la lista de libros ordenada alfabéticamente por título.

Paso 3: Implementar la búsqueda lineal

Imagine que busca un libro concreto en una gran biblioteca. La búsqueda lineal es como revisar sistemáticamente cada libro de una estantería hasta encontrar el que busca. Veamos cómo se traduce esto al código Python.

- Se ha proporcionado la función `search_books`, que implementa un algoritmo de búsqueda lineal.
- Revisa esta función para entender cómo busca libros por título o autor.
- Sustituya el marcador de posición de la variable `query` por "El Gran Gatsby".
- Ejecute la celda para ejecutar la búsqueda lineal y observe los resultados.
- Experimente buscando "Dune" y el autor "J.K. Rowling" y observe los resultados.

Paso 4: Implementar la Búsqueda binaria

Si la biblioteca estuviera ordenada, la búsqueda sería más eficaz. La Búsqueda binaria es como un juego de adivinanzas de alta tecnología, dividiendo repetidamente por la mitad el área de búsqueda hasta encontrar el libro deseado.

- Se ha proporcionado la función `binary_search_books`, que implementa la búsqueda binaria.
- Estudie esta función para comprender su enfoque de Dividir y conquistar.
- Ejecute las celdas que cargan un conjunto de datos mayor ('big_book_catalog.csv') y realice búsquedas lineales y binarias de "El Gran Gatsby".
- Compare los tiempos de ejecución, observando la ventaja de velocidad de la búsqueda binaria, especialmente con conjuntos de datos más grandes.
- Tenga en cuenta que los tiempos exactos variarán en función de la potencia de su sistema, pero espere una búsqueda extremadamente rápida en la búsqueda binaria en comparación con la lineal.
- Ten en cuenta también que los Notebooks de Jupyter pueden "cachear" los resultados de la búsqueda, por lo que ejecutar la misma consulta dos veces puede acabar con tiempos que parezcan cero. Puedes hacer que Jupyter "Reinicie el núcleo" para obtener resultados frescos.

1. ¿Cuál de las siguientes afirmaciones describe mejor la diferencia fundamental entre la búsqueda lineal y la binaria? Seleccione la mejor respuesta.

- ☒ La búsqueda lineal planea comprobar cada elemento de la lista (hasta encontrar el resultado), mientras que la búsqueda binaria planea dividir repetidamente el espacio de búsqueda por la mitad.
- ☐ La Búsqueda binaria siempre es más rápida que la búsqueda lineal, independientemente del tamaño del conjunto de datos.
- ☐ La búsqueda lineal es más rápida que la búsqueda binaria para conjuntos de datos pequeños.
- ☐ La Búsqueda binaria requiere que los datos estén ordenados, mientras que la búsqueda lineal no.

✓ **Correcto**

Correcto Esto describe con exactitud la diferencia algorítmica fundamental entre las dos búsquedas.

2. ¿En qué caso la búsqueda lineal sería más adecuada que la búsqueda binaria?

- ☒ Búsqueda de un valor concreto en una lista pequeña sin ordenar.
- ☐ Buscar una palabra en el diccionario.
- ☐ Búsqueda de un título de libro concreto en un catálogo de biblioteca amplio y ordenado.
- ☐ Encontrar la posición de un elemento concreto en una matriz ordenada.

✓ **Correcto**

Correcto Para conjuntos de datos muy pequeños, la sobrecarga de la ordenación para la búsqueda binaria podría superar sus beneficios, haciendo que la búsqueda lineal sea una opción razonable.

3. Si el conjunto de datos `big_book_catalog.csv` no estuviera ordenado, ¿cómo afectaría esto a su elección del algoritmo de búsqueda?

- ☐ No afectaría a tu elección; seguirías utilizando el mismo algoritmo.
- ☐ Primero se ordena el conjunto de datos y luego se utiliza la búsqueda binaria.
- ☐ De todos modos, usted elegiría la búsqueda binaria, ya que puede trabajar con datos no ordenados.
- ☒ Usted elegiría la búsqueda lineal, ya que la búsqueda binaria requiere que los datos estén ordenados.

✓ **Correcto**

Correcto Sin datos ordenados, la búsqueda binaria no es una opción, y la búsqueda lineal se convierte en la opción por defecto.

4. Imagine que el conjunto de datos `big_book_catalog.csv` crece hasta contener millones de libros. ¿Cómo afectaría esto a la diferencia de rendimiento entre la búsqueda lineal y la binaria?

- ☐ No habría ningún cambio perceptible en la diferencia de rendimiento.
- ☐ La búsqueda lineal sería más rápida que la búsqueda binaria.
- ☐ La diferencia de rendimiento disminuiría, y la búsqueda lineal pasaría a ser casi tan rápida como la binaria.
- ☒ La diferencia de rendimiento sería aún más significativa, ya que la búsqueda binaria es mucho más rápida que la lineal.

✓ **Correcto**

Correcto A medida que aumenta el tamaño del conjunto de datos, la necesidad de la búsqueda lineal de comprobar potencialmente cada elemento consume cada vez más tiempo, mientras que la complejidad temporal logarítmica de la búsqueda binaria se adapta mucho mejor.

Algoritmos y estructuras de datos en Python

Algoritmos

Son como recetas: instrucciones paso a paso para resolver un problema. Están detrás de motores de búsqueda, aplicaciones y casi todo el software que usamos. Permiten transformar problemas complejos en soluciones rápidas y eficientes.

Estructuras de datos

Son la forma de organizar los datos en memoria.

- Una **lista** sirve para mantener elementos en orden.
- Un **diccionario** es ideal para pares clave-valor (ej. nombre y número de teléfono).

La elección correcta de la estructura de datos hace que los algoritmos sean más rápidos y eficientes.

El factor eficiencia: Complejidad

La eficiencia se mide en **tiempo** y **espacio**, lo que en informática se describe con la notación **Big O**.

Ejemplo: ordenar una baraja de cartas.

Ejemplo 1: Algoritmo de ordenación simple

Este método encuentra la carta más baja y la aparta repetidamente. Es fácil de entender, pero lento para listas grandes.

```
def simple_sort(cards):  
    sorted_cards = []  
    while cards:  
        lowest_card = min(cards)  
        sorted_cards.append(lowest_card)  
        cards.remove(lowest_card)  
    return sorted_cards
```

Ejemplo 2: Quicksort (ordenación rápida)

Mucho más eficiente para listas grandes. Utiliza un **pivote** y divide los elementos en menores y mayores, aplicando recursión.

```
def quicksort(cards):
```

```
if len(cards) < 2:

    return cards # Caso base: ya está ordenado si hay 0 o 1
elemento

else:

    pivot = cards[0] # Elegir el primer elemento como pivote

    less = [i for i in cards[1:] if i <= pivot]

    greater = [i for i in cards[1:] if i > pivot]

    return quicksort(less) + [pivot] + quicksort(greater)
```

Conclusión:

Los algoritmos son las instrucciones y las estructuras de datos son los contenedores. Juntos determinan la eficiencia del programa. Conocer la complejidad y elegir el enfoque adecuado (como usar quicksort en lugar de un método ingenuo) es clave para escribir código **rápido y potente** en Python.

Puntos clave sobre Quicksort

- Divide el problema eligiendo un **pivote** y separando la lista en menores/iguales y mayores al pivote.
- Ordena **recursivamente** las sublistas y luego las combina con el pivote.
- En el caso medio, es mucho más rápido que algoritmos sencillos como la ordenación por selección o burbuja, sobre todo con **grandes volúmenes de datos**.
- No siempre es el más adecuado: la elección del algoritmo depende de las **características de los datos** y del caso de uso.

Notación Big O

- Mide cómo cambia el **rendimiento de un algoritmo** al aumentar el tamaño de los datos.

- Ejemplo: *si duplicas el tamaño de la lista, el tiempo puede aumentar al doble, al cuádruple o más, según el algoritmo.*
- Dominar Big O permite **predecir eficiencia** y escribir código escalable.

Perspectivas opuestas: las compensaciones

- Algunos prefieren algoritmos **simples** y fáciles de implementar aunque sean lentos.
- Otros priorizan la **velocidad** y usan algoritmos más complejos.
- No hay una única respuesta correcta: depende de las prioridades (simplicidad vs rendimiento).

Aplicaciones reales de los algoritmos

- **GPS y navegación** → usan algoritmos de grafos para calcular rutas rápidas.
- **Streaming (Netflix, Spotify)** → algoritmos de **aprendizaje automático** para recomendaciones.
- **Imágenes médicas** → algoritmos de procesamiento que ayudan a detectar enfermedades en escáneres.
- También están en **detección de fraudes, optimización industrial y descubrimiento de fármacos.**

El viaje hacia la maestría

- Dominar algoritmos y estructuras de datos permite resolver problemas reales con eficiencia.
- Entender y elegir los **bloques de construcción adecuados** en cada proyecto es lo que diferencia a un programador avanzado.

En resumen:

Quicksort es un ejemplo de cómo un buen algoritmo puede ser mucho más eficiente que

los básicos. La notación Big O ayuda a medir esta eficiencia. Según el contexto, se deben elegir algoritmos simples o complejos. Los algoritmos no son solo teoría: están presentes en navegación, entretenimiento, medicina y más. Aprender a dominarlos es el camino hacia la excelencia en Python.

Cuestionario

1. ¿Cuál de las siguientes afirmaciones es cierta sobre los algoritmos de búsqueda lineal y binaria? Seleccione todas las que correspondan.

☒ La Búsqueda binaria requiere ordenar el conjunto de datos antes de poder utilizarlo eficazmente.

☒ **Correcto**

Correcto La Búsqueda binaria requiere que los datos estén ordenados para funcionar correctamente.

☐ La búsqueda lineal es más rápida que la búsqueda binaria para conjuntos de datos grandes y ordenados.

☒ La búsqueda lineal puede utilizarse tanto en conjuntos de datos ordenados como sin ordenar.

☒ **Correcto**

Correcto La búsqueda lineal no requiere que el conjunto de datos esté ordenado.

☒ La Búsqueda binaria suele ser muy eficaz para buscar dentro de grandes conjuntos de datos que ya están ordenados.

☒ **Correcto**

Correcto La Búsqueda binaria es efectivamente muy eficiente para datos ordenados.

2. Tienes una lista de nombres ordenados alfabéticamente y necesitas encontrar un nombre concreto muy rápidamente. ¿Qué método de búsqueda es el más rápido para esta tarea? Selecciona la mejor respuesta.

☐ Búsqueda lineal, que comprueba cada nombre uno a uno

☐ Un método de clasificación, que ordena la lista antes de buscarla.

☒ Búsqueda binaria, que acota rápidamente la búsqueda.

☐ Un método que busca nombres al azar hasta encontrar el correcto

☒ **Correcto**

Correcto La Búsqueda binaria es la más rápida para las listas ordenadas porque acota rápidamente la búsqueda.

3. Un desarrollador de software está optimizando una gran plataforma de comercio electrónico que clasifica productos para los clientes. La plataforma debe gestionar miles de productos y clasificarlos rápidamente en función de varios criterios. En este contexto, ¿a qué se refiere principalmente la "eficiencia"? Seleccione la mejor respuesta.

- ☐ Capacidad del algoritmo para gestionar descripciones complejas de productos
- ☐ Legibilidad del algoritmo
- ☒ El consumo de tiempo y recursos de memoria del algoritmo al clasificar los productos
- ☐ Compatibilidad del algoritmo con distintos navegadores web

✓ **Correcto**

Correcto En el comercio electrónico, eficiencia significa clasificación rápida con una carga mínima del servidor.

4. Usted está diseñando un sistema de biblioteca en el que los libros se almacenan en estanterías en orden alfabético por título. Un usuario quiere encontrar un libro concreto. ¿Qué algoritmo de búsqueda utilizaría instintivamente el bibliotecario? Seleccione la mejor respuesta.

- ☐ Búsqueda lineal - Empezando por el primer libro y comprobando cada uno de ellos
- ☐ Búsqueda binaria - Es el algoritmo de búsqueda más complejo
- ☐ Búsqueda lineal - Es la única forma de buscar libros
- ☒ Búsqueda binaria - Dividir repetidamente las estanterías por la mitad

✓ **Correcto**

Correcto El bibliotecario probablemente utilizaría una variación de la búsqueda binaria, acotando el área de búsqueda en función del orden alfabético

5. Un desarrollador está creando una aplicación de redes sociales en la que las publicaciones de los usuarios deben mostrarse en un feed basado en una combinación de relevancia y actualidad. Para gestionar y mostrar eficientemente estos datos, ¿cuál es el papel principal de las "estructuras de datos" en este escenario? Seleccione la mejor respuesta.

- ☒ Organizar los datos de los POST de forma que puedan recuperarse y clasificarse rápidamente en función de la relevancia y el tiempo
- ☐ Utilizar la Notación O mayúscula para evaluar la velocidad de carga de la aplicación para diferentes usuarios
- ☐ Para resumir los pasos específicos que sigue la aplicación para clasificar y mostrar los mensajes
- ☐ Para seleccionar el lenguaje de programación más popular para el backend de la aplicación

✓ **Correcto**

Correcto Las estructuras de los datos son fundamentales para la gestión eficaz de los datos en aplicaciones dinámicas.

Elegir la estructura de datos adecuada: Un marco de decisión

Qué son

- Son **contenedores** que organizan, almacenan y permiten manipular datos en memoria.
- Cada estructura está diseñada con un **propósito específico**: rapidez de acceso, orden, gestión de grandes volúmenes, etc.

Por qué son importantes

- Mejoran la **eficiencia** del código en velocidad y uso de memoria.
- Aumentan la **legibilidad** y facilitan el mantenimiento.
- Son clave para aplicaciones como motores de búsqueda, tiendas online o redes sociales.

Factores para elegir la estructura adecuada

1. **Tipo de datos** → numéricos, textuales, con relaciones o jerarquías.
2. **Operaciones necesarias** → búsqueda, ordenación, inserción, eliminación, iteración.
3. **Rendimiento** → rapidez en búsquedas, frecuencia de actualizaciones, tamaño del dataset.
4. **Restricciones de memoria** → optimización según el entorno.
5. **Legibilidad y mantenimiento** → claridad para otros desarrolladores (o tu yo futuro).

Ejemplos de uso

- **Diccionarios:** para recomendaciones personalizadas en tiendas online (clave = ID producto, valor = detalles).
- **Listas:** para feeds de redes sociales, ya que mantienen el orden cronológico.
- **Sets:** para inventarios de almacenes, evitando duplicados.

En resumen:

Las estructuras de datos son la base para organizar información en Python. Elegir la correcta —considerando tipo de datos, operaciones, rendimiento, memoria y legibilidad— mejora la velocidad, claridad y eficacia del código.

Pregunta

Estás desarrollando un programa en Python para gestionar el directorio de empleados de una empresa, donde necesitas almacenar y acceder rápidamente a la información de los empleados basándote en sus números de identificación únicos. ¿Qué estructura de datos sería la opción MÁS eficiente para este escenario? Selecciona la mejor respuesta.

- ☐ Una lista anidada, en la que cada lista interior representa a un empleado y sus datos se almacenan como elementos individuales dentro de la lista interior.
- ☒ Un diccionario, utilizando los ID de los empleados como claves y su información detallada (nombre, departamento, datos de contacto) como valores.
- ☐ Un conjunto de identificadores de empleados, que garantiza la unicidad y permite comprobar rápidamente la afiliación.
- ☐ Una lista de tuplas, donde cada tupla contiene un ID de empleado y su información correspondiente.

Correcto

¡Correcto! Los diccionarios son excelentes para realizar búsquedas rápidas basadas en claves, lo que permite una recuperación eficaz de la información de los empleados utilizando sus identificadores únicos.

Estudios de caso: Adecuación de las estructuras de datos a los problemas

Idea principal

Las estructuras de datos (listas, diccionarios y conjuntos) son esenciales para organizar la información, mejorar el rendimiento y resolver problemas reales de forma eficiente.

Casos de estudio

1. **Motor de recomendaciones de comercio electrónico**
 - **Estructura usada:** Diccionarios.

- **Cómo funciona:** claves = ID de cliente, valores = lista de productos recomendados.
- **Resultado:** recomendaciones rápidas y personalizadas → mayor participación, ventas y fidelidad.

2. Temas de tendencias en redes sociales

- **Estructuras usadas:** Listas y conjuntos.
- **Cómo funciona:** listas para guardar publicaciones en orden cronológico; conjuntos para eliminar duplicados y detectar hashtags únicos.
- **Resultado:** identificación eficiente de tendencias en tiempo real → más conexión y participación de los usuarios.

3. Sistema de detección de fraudes

- **Estructuras usadas:** Listas y diccionarios.
- **Cómo funciona:** listas para almacenar transacciones en orden; diccionarios para patrones y estadísticas de cada cliente.
- **Resultado:** detección temprana de anomalías → reducción de fraudes y mayor seguridad.

Conclusión

- Elegir la **estructura adecuada** afecta directamente la **eficiencia, legibilidad y éxito** de un proyecto.
- Comprender las fortalezas y debilidades de listas, diccionarios y conjuntos permite tomar **decisiones informadas**.
- La práctica y la experimentación son claves para desarrollar soluciones más potentes e innovadoras en Python.

Pregunta

Estás trabajando en un proyecto que requiere que mantengas una clasificación en tiempo real de los hashtags más populares en una plataforma de Redes sociales. ¿Qué combinación de estructuras de datos sería la MÁS adecuada para almacenar y actualizar eficientemente esta información? Selecciona la mejor respuesta.

- ☐ Un diccionario, donde los hashtags son claves y sus puntuaciones de popularidad son valores, junto con un conjunto para rastrear hashtags únicos.
- ☐ Un conjunto de hashtags para garantizar la unicidad, combinado con una lista ordenada para mantener la clasificación en tiempo real basada en la popularidad.
- ☐ Una lista de hashtags, ordenados alfabéticamente, junto con un diccionario independiente para almacenar sus puntuaciones de popularidad.
- ☒ Una lista para almacenar los hashtags entrantes, convertidos periódicamente en un conjunto para eliminar duplicados, luego contados y clasificados para actualizar la lista de tendencias.

✔ **Correcto**

Correcto ASÍ SE ALINEA CON EL CASO DE ÉXITO DE TENDENCIAS EN LAS REDES SOCIALES DEL GUIÓN. La lista gestiona eficazmente los datos entrantes, el conjunto garantiza la unicidad y el recuento/ranking periódico actualiza la lista de tendencias.

Trucos y consejos para la selección de la estructura de datos

Idea principal

Elegir la estructura de datos adecuada es clave para escribir código rápido, claro y eficiente en Python. Cada estructura (lista, diccionario, conjunto, tupla) tiene sus ventajas y limitaciones, y la elección depende del tipo de datos, operaciones, tamaño y requisitos de rendimiento.

Estructuras básicas en Python

- **Listas** → Ordenadas, mutables, versátiles (dinámicas).
- **Diccionarios** → Pares clave-valor, búsquedas rápidas.
- **Conjuntos** → Elementos únicos, sin orden, ideales para eliminar duplicados.
- **Tuplas** → Ordenadas, inmutables, útiles para datos fijos.

Factores clave para seleccionar

1. **Tipo de datos**

- Ordenados: listas, tuplas.
- Desordenados: conjuntos.
- Mutables: listas, diccionarios.
- Inmutables: tuplas.

2. Operaciones más frecuentes

- Añadir/eliminar: listas (append/pop), conjuntos y diccionarios (eficientes).
- Búsquedas: diccionarios (claves) y conjuntos (pertenencia).
- Ordenar: listas.

3. Tamaño de los datos

- Pequeños: elige lo más cómodo.
- Grandes: importa la eficiencia (listas más lentas que diccionarios en búsquedas).

4. Complejidad temporal y espacial

- Tiempo: p. ej. búsqueda en lista desordenada = $O(n)$, en diccionario = $O(1)$.
- Espacio: diccionarios consumen más memoria, pero aceleran las búsquedas.

Escenarios prácticos

1. **Lista dinámica de reproducción de música** → **Lista** (orden, mutabilidad, métodos append/pop).
2. **Búsquedas rápidas por ID único (usuarios en red social)** → **Diccionario** (claves únicas, búsquedas $O(1)$).
3. **Eliminar duplicados de correos** → **Conjunto** (unicidad automática).
4. **Guardar coordenadas fijas** → **Tupla** (ordenadas, inmutables, protegen integridad).

Conclusión

La mejor estructura depende del equilibrio entre tipo de datos, operaciones necesarias, tamaño, rendimiento y legibilidad. Comprender estas diferencias permite tomar decisiones inteligentes y escribir código más eficaz y mantenible.

Comparación de búsqueda en listas y diccionarios

Se muestra cómo la búsqueda en diccionarios es mucho más rápida que en listas, especialmente para grandes conjuntos de datos:

```
import timeit

# Búsqueda en lista

list_data = list(range(100000))

lookup_value = 99999

list_time = timeit.timeit(lambda: lookup_value in list_data,
number=1000)

# Búsqueda en diccionario

dict_data = {i: i for i in range(100000)}

dict_time = timeit.timeit(lambda: lookup_value in dict_data,
number=1000)

print("List lookup time:", list_time)

print("Dictionary lookup time:", dict_time)
```

Resultado típico:

```
List lookup time: 3.054272091
```

```
Dictionary lookup time: 0.000176368
```

La búsqueda en diccionarios es significativamente más rápida debido a su estructura basada en hash, mientras que las listas requieren recorrer los elementos secuencialmente.

Estructuras de datos especializadas en Python

1. Deque (cola de doble extremo)

Permite añadir o eliminar elementos eficientemente desde ambos extremos. Ideal para colas (como las de un parque de atracciones que se añaden gente al final de la fila y se montan los primeros de la fila), pilas, simulaciones de líneas o funcionalidad de deshacer/rehacer.

```
from collections import deque

queue = deque()

queue.append("task1")

queue.append("task2")

print(queue.popleft()) # Output: task1
```

2. Heapq (cola prioritaria)

Mantiene el elemento con la prioridad más alta siempre al frente. Útil en programación de tareas, algoritmos de caminos más cortos (como el algoritmo de Dijkstra), o sistemas basados en prioridades.

Ejemplo conceptual: sala de urgencias donde los pacientes más graves son atendidos primero.

3. collections.Counter

Cuenta la frecuencia de elementos en un iterable. Perfecto para analizar palabras, contar elementos de listas, o identificar los más comunes.

```
from collections import Counter

text = "This is a sample text with some repeated words words"
```

```
word_counts = Counter(text.split())

print(word_counts)

# Output: Counter({'words': 2, 'This': 1, 'is': 1, 'a': 1, 'sample':
1, 'text': 1, 'with': 1, 'some': 1, 'repeated': 1})
```

Conclusión

- Elegir la estructura de datos correcta depende del **tipo de operación y escenario de uso**.
- Diccionarios son rápidos para búsquedas, listas son útiles para secuencias, deques permiten operaciones eficientes en ambos extremos, heapq gestiona prioridades, y Counter facilita conteos.
- No existe una elección universal; comprender las fortalezas de cada estructura permite escribir código eficiente y legible.

Cuestionario

1. Usted está construyendo un sistema que necesita almacenar una secuencia de IDs de transacciones en el orden exacto en que ocurrieron. Esta lista deberá actualizarse con el tiempo. ¿Qué estructura de los datos es la MÁS adecuada para esta tarea? Seleccione la mejor respuesta.

- ☐ Fije
- ☒ Lista
- ☐ Tupla
- ☐ Diccionario

✓ **Correcto**

Correcto Las listas mantienen el orden de los elementos, lo que es esencial para almacenar los ID de transacción en la secuencia en que se produjeron.

2. Está construyendo un sistema para almacenar información de contacto de clientes, donde cada cliente se identifica por una dirección de correo electrónico única. ¿Qué estructura de los datos sería la MÁS eficaz para recuperar rápidamente la información de un cliente utilizando su correo electrónico? Elija la mejor respuesta.

- ☒ Diccionario
- ☐ Tuplas
- ☐ Fije
- ☐ Lista

✓ **Correcto**

Correcto. Los diccionarios permiten realizar búsquedas rápidas mediante claves (direcciones de correo electrónico), por lo que resultan ideales para recuperar información sobre clientes de forma eficaz

3. Tiene una colección de respuestas a una encuesta con respuestas de texto abierto. Desea averiguar qué palabras se utilizaron en todas las respuestas y cuántas veces aparecen. ¿Qué estructura de los datos sería la más sencilla para almacenar y contar el número de apariciones de las palabras? Seleccione la mejor respuesta.

- ☐ Lista
- ☒ Diccionario
- ☐ Fije
- ☐ Tupla

✓ **Correcto**

Correcto Puede utilizar un diccionario en el que cada palabra única sea una clave y el valor sea su recuento. Esto permite un seguimiento eficiente de las palabras únicas y sus frecuencias.

4. Es necesario almacenar una secuencia de cadenas en el orden exacto en que se añaden. Este orden es importante, y la colección de cadenas deberá modificarse con el tiempo (es posible que se añadan o eliminen cadenas). ¿Qué estructura de los datos es la MÁS adecuada para esta tarea? Seleccione la mejor respuesta.

- ☐ Fije
- ☐ Diccionario
- ☒ Lista
- ☐ Tupla

✓ **Correcto**

Correcto Las listas mantienen el orden en el que se añaden los elementos, y son mutables, lo que significa que puedes añadir, eliminar o modificar fácilmente la secuencia de cadenas a lo largo del tiempo.

5. Un desarrollador está trabajando en un proyecto que requiere almacenar una colección de elementos únicos. ¿Qué estructura de los datos es la más adecuada para esta situación? Seleccione la mejor respuesta.

- ☐ Array
- ☒ Fije
- ☐ Pila
- ☐ Cola

sets

✓ **Correcto**

Correcto Los conjuntos están diseñados para almacenar elementos únicos y son perfectos para este escenario.

Actividad final: Estructuras de datos en Python

1. ¿Qué estructura de datos de Python es la más adecuada para almacenar una colección ordenada de elementos que pueden tener que modificarse posteriormente? Seleccione la mejor respuesta.

- ☒ Lista
- ☐ Fije
- ☐ Diccionario
- ☐ Tupla

✓ **Correcto**

Correcto Las listas son colecciones versátiles y ordenadas que permiten añadir, eliminar y modificar elementos, lo que las hace ideales para el almacenamiento dinámico de datos.

2. Estás organizando una aplicación de recetas, y quieres almacenar una lista de ingredientes para cada receta, donde el orden de los ingredientes es importante para el proceso de cocción. ¿Qué estructura de los datos de Python elegirías para esta tarea? Selecciona la mejor respuesta.

- ☐ Diccionario
- ☐ Tupla
- ☐ Fije
- ☒ Lista

✓ **Correcto**

Correcto Las listas son colecciones ordenadas, lo que las hace perfectas para almacenar ingredientes en la secuencia necesaria para la receta.

1. ¿Cuál de las siguientes situaciones pone de manifiesto las ventajas de utilizar estructuras de datos mutables en Python? Seleccione todas las que correspondan.

☒ Estás desarrollando un juego en el que la puntuación del jugador debe actualizarse a medida que avanza por los niveles.

☒ **Correcto**

Correcto Las estructuras mutables como variables o diccionarios pueden almacenar y actualizar fácilmente la puntuación del jugador a medida que gana puntos.

☐ Estás almacenando las coordenadas de un punto de referencia en un mapa, y estas coordenadas deben permanecer fijas y sin cambios durante la ejecución de la aplicación.

☒ Estás creando una aplicación de chat en tiempo real en la que es necesario añadir nuevos mensajes al historial de conversaciones a medida que se envían y reciben.

☒ **Correcto**

Correcto Las estructuras mutables como las listas permiten añadir dinámicamente nuevos mensajes, reflejando la conversación en curso.

☒ Estás creando un programa para calcular la temperatura media a lo largo de una semana, donde se recogen y almacenan las temperaturas diarias.

☒ **Correcto**

Correcto Una estructura mutable como una lista permite añadir nuevas lecturas de temperatura cada día y, a continuación, realizar cálculos sobre los datos recopilados.

4. Usted tiene un diccionario que representa los datos de los empleados de una empresa, donde el nombre de cada empleado es una clave, y sus detalles (como departamento, salario, etc.) se almacenan como valores dentro de diccionarios anidados. ¿Cuál de las siguientes tareas puede realizar iterando sobre las claves y valores de este diccionario? Seleccione todas las que correspondan.

☒ Generar un informe con la lista de todos los empleados y sus departamentos

☒ **Correcto**

Correcto Iteración le permite acceder al nombre y departamento de cada empleado, permitiéndole crear el informe deseado.

☒ Calcular el salario medio de todos los empleados

☒ **Correcto**

Correcto Al iterar por el diccionario, puede acceder al salario de cada empleado y calcular la media.

☐ Añadir un nuevo empleado al diccionario

☒ Identificar a los empleados que pertenecen a un departamento específico

☒ **Correcto**

Correcto Puede comprobar el departamento de cada empleado dentro del diccionario anidado y filtrar en función del departamento deseado.

Añadir un nuevo empleado al diccionario **no se hace iterando**, sino asignando un nuevo par clave-valor directamente.

5. Está diseñando un sistema de gestión de tareas en el que éstas deben procesarse en el orden en que se reciben. ¿Qué estructura de los datos es la más adecuada para almacenar estas tareas? Seleccione la mejor respuesta.

- ☐ Pila
- ☐ Fije
- ☐ Diccionario
- ☒ Cola

✓ **Correcto**

Correcto Las colas siguen el principio de Primero en entrar, primero en salir (FIFO), lo que garantiza que las tareas se procesan en el orden en que llegan.

6. Tienes una lista de nombres de alumnos y quieres ordenarlos alfabéticamente. ¿Qué función de Python utilizarías para conseguirlo? Selecciona la mejor respuesta.

- ☐ list.reverse()
- ☒ lista.ordenar()
- ☐ list.append()
- ☐ list.index()

✓ **Correcto**

Correcto El método `list.sort()` ordena los elementos de una lista en su lugar, modificando la lista original.

7. Necesitas encontrar la posición de la primera aparición de un valor específico en una lista. ¿Qué método de lista de Python utilizarías? Selecciona la mejor respuesta.

- ☐ list.count()
- ☐ list.append()
- ☒ list.index()
- ☐ lista.ordenar()

✓ **Correcto**

Correcto El método `list.index()` devuelve el índice de la primera aparición del valor especificado en la lista.

8. Estás implementando una función de búsqueda en una aplicación de biblioteca musical. La biblioteca contiene una amplia colección de canciones y los usuarios necesitan poder encontrarlas rápidamente por sus títulos. Teniendo en cuenta el tamaño y la naturaleza de los datos, ¿qué algoritmo de búsqueda sería el más adecuado si los títulos de las canciones estuvieran ordenados alfabéticamente? Seleccione la mejor respuesta.

- ☐ Búsqueda lineal
- ☒ Búsqueda binaria
- ☐ Clasificación simple
- ☐ Ordenación rápida

✓ **Correcto**

Correcto Con un conjunto de datos grande y ordenado, la búsqueda binaria ofrece una eficacia superior a la búsqueda lineal, lo que la convierte en la opción ideal para realizar búsquedas rápidas en la biblioteca musical.

9. Usted tiene la tarea de crear un programa para almacenar la información de los estudiantes, incluyendo sus nombres, IDs y calificaciones. Necesitas una estructura de datos que permita un acceso rápido a la información de un estudiante basándose en su ID. ¿Qué estructura de los datos sería la más adecuada para esta tarea? Seleccione la mejor respuesta.

- ☒ Diccionario
- ☐ Tupla
- ☐ Lista
- ☐ Fije

✓ **Correcto**

¡Correcto! Los diccionarios son ideales para realizar búsquedas rápidas mediante claves, lo que los convierte en la opción perfecta para almacenar información de los alumnos y acceder a ella rápidamente por ID.

10. Está trabajando con coordenadas geográficas (latitud, longitud) que representan ubicaciones fijas en un mapa. Es crucial que estas coordenadas permanezcan inmutables una vez establecidas. ¿Qué estructura de los datos garantizaría mejor la inmutabilidad de estas coordenadas? Selecciona la mejor respuesta.

- ☒ Tupla
- ☐ Diccionario
- ☐ Lista
- ☐ Fije

✓ **Correcto**

Correcto Las tuplas están ordenadas y son inmutables, lo que garantiza que los valores de latitud y longitud no puedan modificarse accidentalmente después de haber sido establecidos inicialmente.